

5.4 Solutie posibila #2

5.4.1 Descrierea modului hardware

Vom prezenta in continuare o implementare a unui sistem avand la baza placa de dezvoltare cu microcontrolerul C8051F040. Acesteia i se adauga doua module externe:

- modulul de intrare
 - o contine o celula solara (dispozitivul de achizitie) ce se comporta ca o sursa de tensiune variabila (tensiunea oferita la iesirea celulei solare este direct proportionala cu intensitatea luminii incidente).
 - o se conecteaza la microcontroler prin doua fire; unul este masa sistemului, iar celalalt este pentru tensiunea de iesire a celulei solare.
 - o schema modului este prezentata in Fig. 5.4.1.a.

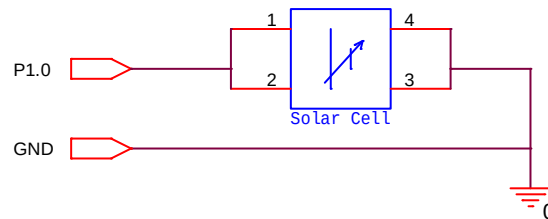


Fig. 5.4.1.a. Modulul de intrare

- modulul de iesire
 - o contine, printre alte componente ce nu fac obiectul acestei lucrari, un dispozitiv de afisare (led bicolor rosu-verde).
 - o se conecteaza la microcontroler printr-o magistrala cu 10 fire; dintre acestea sunt folosite doar trei: masa sistemului, anodul "rosu" al led-ului, anodul "verde" al ledului. Observatie: led-ul bicolor este un sistem ce incorporeaza in aceeasi capsula doua led-uri de culori diferite care impart acelasi catod.
 - o Schema partii de modul ce priveste acest laborator este prezentata in figura Fig. 5.4.1.b.

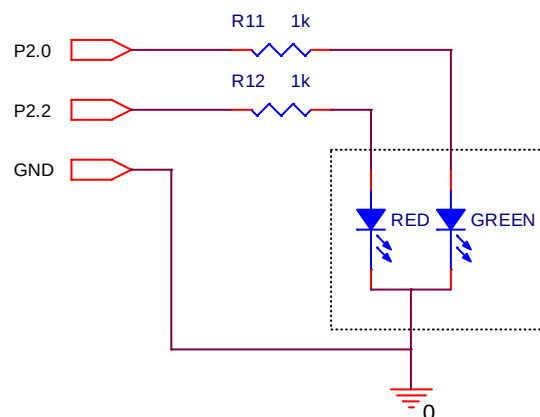


Fig. 5.4.1.b. Modulul de iesire

Dupa cum am spus celula solara se comporta ca o sursa de tensiune variabila cu valori intre 0 si 2.4 volti. Aceasta tensiune este introdusa in multiplexorul analogic prin pinul P1.0, modul de achizitie fiind simplu (nu diferential).

Valoarea digitala de la iesirea sistemului de conversie analog-digitala este data de formula:

$$\text{Cod} = V_i * (K/V_{\text{ref}}) * 256$$

In care:

- Cod este pe opt biti (intre 0 - 255 in zecimal)
- V_i este tensiunea de la intrarea multiplexorului analogic

- K este castigul amplificatorului programabil (0.5, 1, 2, 4)
- V_{ref} este tensiunea de referinta

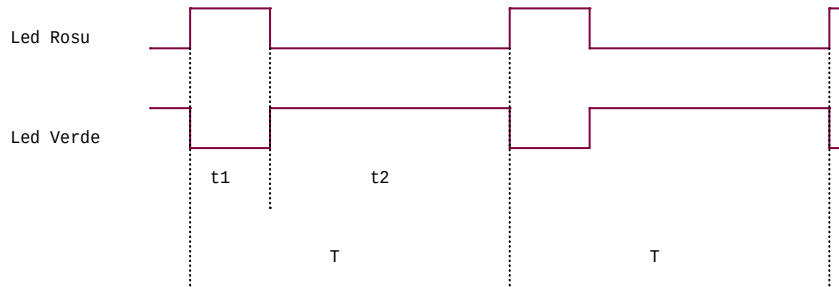
Observatie: codul convertit este un numar intre 0 si 255, deci $V_i * (K/V_{ref})$ trebuie sa aiba o valoare subunitara.

In aceste conditii se alege tensiunea de referinta V_{ref} egala cu 2.4V (se foloseste chiar referinta interna de tensiune a microcontrolerului), iar castigul amplificatorului programabil K egal cu 1. Pentru a obtine 2.4V la iesirea referintei interne de tensiune trebuie activat si amplificatorul de castig 2 (vezi capitolul 2.1.3.). Pentru a ruta referinta de tensiune pinii 5 si 6 din J22 trebuie conectati printr-un jumper.

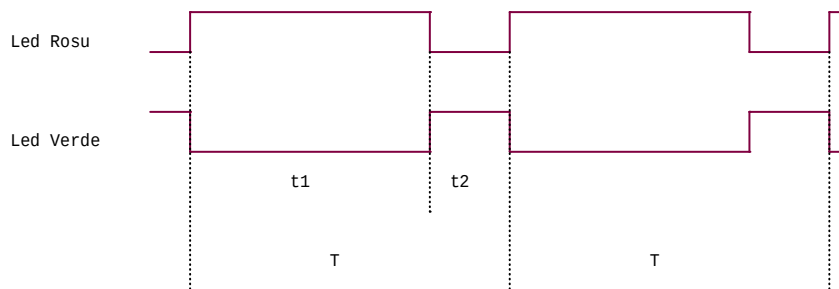
La iesirea sistemului se conecteaza pe pinii P2.0 si P2.2 un led bicolor. In mod normal un astfel de dispozitiv de afisare se poate afla in una din urmatoarele patru stari: stins complet, aprins rosu, aprins verde, aprins atat rosu cat si verde (obtinandu-se o culoare intermediara). Totusi, in aplicatia de fata s-a gasit o modalitate de afisare calitativa (cu mai multe stari) variind factorul de umplere al semnalelor aplicate celor doua led-uri.

Dupa cum ati vazut in lucrarea trecuta, sistemul vizual uman nu percepe palparea unui led cu o frecventa mare (mai mare decat frecventa critica). Acest fenomen se exploateaza si in aceasta lucrare, perioada semnalelor aplicate led-urilor fiind foarte scurta.

Intr-o astfel de perioada led-ul rosu se va aprinde un interval de timp t_1 , iar led-ul verde un interval de timp t_2 . Raportul acestor intervale dicteaza culoarea perceputa de sistemul vizual uman. Exemple:



Exemplul 1



Exemplul 2

In Exemplul 1, culoarea perceputa de sistemul vizual uman este foarte aproape de verde, cu o nuanta rosiatica, in timp ce in Exemplul 2, rosul este predominant.

Observatie: este necesar ca perioada T sa fie constanta.

Folosind aceasta metoda s-a ales sa se imparta perioada semnalului T in 16 cuante egale, led-ul bicolor putand sa se aprinda pe 16 nivele de culoare intre rosu si verde (verde pur va indica intuneric, iar rosu pur iluminare puternica).

5.4.2 Descrierea algoritmului

Observatie: programul respecta structura generala prezentata in Lucrarea nr.1, capitolul 3.2.2.

In solutia propusa in acest laborator programul va avea in mai multe rutine: `main`, `baseDelay`, `init`, `lightTest`, `redOn`, `redOff`, `greenOn`, `greenOff`.

Algoritmul consta intr-o bucla de durata constanta ce se autoapeleaza la infinit. Aceasta bucla are trei etape:

- se converteste tensiunea de la bornele celulei solare intr-o valoare digitala pe opt biti (intre 0 si 255).
- se imparte valoarea pe opt biti la 16 obtinandu-se un numar intre 0 si 15, reprezentand chiar nivelul de culoare
- se aprinde led-ul rosu un interval de timp (t_1) determinat de nivelul de culoare.
- se aprinde led-ul verde un interval de timp (t_2) determinat de nivelul de culoare.

Observatie: durata buclei este constanta, deci suma intervalelor de timp cat sunt aprinse ledurile ($T = t_1 + t_2$) este de asemenea constanta. Aceasta perioada este impartita in 16 cuante egale.

Subrutina `lightTest` face o conversie analog-digitala. Valoarea rezultata (intre 0 si 255) este impartita la 16, obtinandu-se un numar intre 0 si 15, reprezentand numarul de cuante din perioada semnalului cat va sta aprins led-ul rosu. Acest numar este stocat in registrul R6. Registrul R7 va stoca numarul de cuante cat va sta aprins led-ul verde ($15 - R6$).

Subrutina `baseDelay` este o rutina de intarziere care foloseste registrul R2 pe post de contor.

Rutina principala (`main`) apeleaza subrutina de initializare (`init`), apoi trece in zona de bucla infinita etichetata `start`. Aici se apeleaza rutina `lightTest` folosind apoi valorile stocate in R6 si R7. Codul etichetat `r` aprinde led-ul rosu daca este necesar (daca numarul de cuante cat trebuie sa stea aprins este nenul). Se executa apoi codul etichetat `redLoop` in care led-ul rosu este tinut aprins un interval $R6 * baseDelay$, dupa care este stins. In mod analog se parcurge codul etichetat `g`, apoi `greenLoop`, care va aprinde led-ul verde un interval $R7 * baseDelay$. In final, se executa un salt neconditionat la eticheta `start`.

Subrutina de initializare

- dezactiveaza intreruperile
 - o pentru detalii, a se vedea explicatiile din Lucrarea nr. 1.
- dezactiveaza watchdog timer-ul
 - o pentru detalii, a se vedea explicatiile din Lucrarea nr. 1.
- activeaza crossbar-ul
 - o pentru detalii, a se vedea explicatiile din Lucrarea nr. 1.
- configureaza portul P1
 - o aplicatia configureaza pinul 0 din portul P1 ca fiind pin de intrare analogica pentru convertorul analog-digital. Configurarea modului de intrare al portului se face modificand valoarea stocata in registrul P1MDIN.
 - o definitia registrului P1MDIN se afla in documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 219).
- configureaza portul P2
 - o aplicatia foloseste pinul 0 si pinul 2 din portul P2 pentru a comanda cele 2 led-uri (rosu si verde). Deci, pinul P2.0 si pinul P2.2 trebuie setati ca pini digitali de iesire. Modul de iesire va fi Push-Pull. Configurarea modului de iesire al portului se face modificand valoarea stocata in registrul P2MDOUT.
 - o observatie: pentru a nu afecta celelalte circuite conectate cu panglica, toti pinii portului sunt rutati initial la masa.
 - o definitia registrului P2MDOUT se afla in documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 218).
- configureaza tensiunea de referinta
 - o referinta de tensiune interna, la care este conectat convertorul analog-digital ADC2, este activata prin setarea bitilor BIAS (activeaza generatorul) si REFBE (activeaza bufferul referintei de tensiune) din registrul REF0CN.
 - o definitia registrului REF0CN se afla in documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 114).
- configureaza convertorul analog-digital
 - o pinul P1.0 este selectat ca intrare single-ended in multiplexorul analogic
 - o convertorul se configureaza pentru a porni conversia la setarea bitului AD2BUSY si pentru a avea castigul 1.

Organigrama programului este prezentata in Fig. 5.4.2.

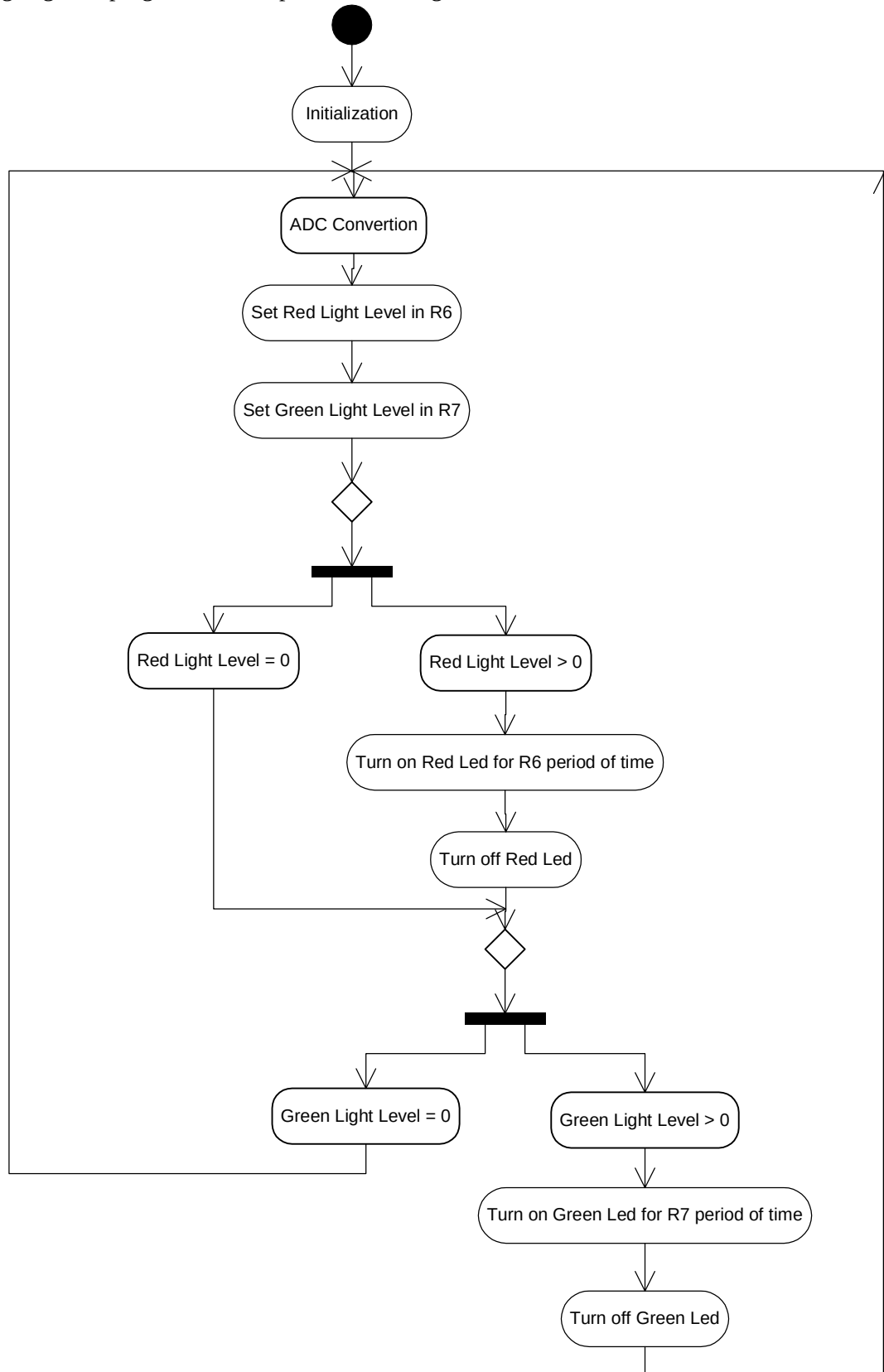


Fig. 5.4.2. Organigrama aplicatiei RedGreenLed

5.5 Codul aplicatiei RedGreenLed

```

#include (c8051f040.inc)                ; Include register definition file.

;-----
; EQUATES
;-----

        RED        equ        P2.2
        GREEN      equ        P2.0

;-----
; RESET and INTERRUPT VECTORS
;-----

                cseg        AT 0x0000        ; Reset Vector
                ljmp        main            ; Locate a jump to the start of code
                                           ; at the reset vector.
;-----
; MAIN PROGRAM CODE SEGMENT
;-----

mainCodeSeg    segment        CODE

                rseg        mainCodeSeg      ; Switch to this code segment.
                using      0                ; Specify register bank for the following
                                           ; program code.

main:
                acall      init              ; Initialization of all used SFR's
                mov        SFRPAGE, #ADC2_PAGE ; Use SFRs on the ADC2 Page

start:
                                           ; Starting the algorithm

                acall      lightTest         ; Tests the intensity of the light

        r:
                mov        A, R6
                jz          g
                acall      redOn
        redLoop:
                acall      baseDelay         ; Keeps the red LED lighted for
                dec        A                ; R6 base_delays
                jnz        redLoop
                acall      redOff           ; Red loop is over. Turn off the red LED

        g:
                mov        A, R7
                jz          start
                acall      greenOn
        greenLoop:
                acall      baseDelay         ; Keeps the green LED lit for
                dec        A                ; R7 base_delays
                jnz        greenLoop
                acall      greenOff         ; Green loop is over. Turn off the green LED

                sjmp       start            ; Start over again

;-----
; FUNCTION CODE
;-----

init:
initWatchDogTimer:    ...                ; Disable global interrupts
                                           ; Disable WatchDog Timer

initCrossbar:        ...                ; Enable Crossbar

initIOPorts:
                anl        P1MDIN, #0xFE    ; Configure P1.0 as analog input
                orl        P2MDOUT, #0x05   ; Set P2.0 and P2.2 (GREEN/RED) as digital
                                           ; output in push-pull mode.
                mov        P2, #0x00        ; Route P2 to ground

initVREF:            mov        SFRPAGE, #0x00 ; Use SFRs on the 0x00 Page

```

```

                                mov        REF0CN, #0x03        ; ADC2 voltage reference from internal VREF
                                                                ; Enable Bias Generator

initADC2:                      mov        SFRPAGE, #ADC2_PAGE    ; Use SFRs on the ADC2 Page
                                mov        AMX2CF, #0x00        ; Set AIN1.0 as single-ended input
                                mov        AMX2SL, #0x00        ; Select AIN1.0 as input channel for
                                                                ; the analog multiplexer
                                mov        ADC2CF, #0xF9        ; Configure a*1 Gain
                                mov        ADC2CN, #0x80        ; Enable ADC2; Continuous tracking;
                                ret

baseDelay:                     mov        R2, #0x0F
                                djnz       R2, $
                                ret

redOn:                          ...                            ; Turn on the Red LED

redOff:                         ...                            ; Turn off the Red LED

greenOn:                        ...                            ; Turn on the Green LED

greenOff:                       ...                            ; Turn off the Green LED

lightTest:                     clr        AD2INT
                                setb      AD2BUSY                ; Force ADC2 to start a conversion
                                jnb       AD2INT, $              ; Wait for ADC2 to finish the conversion

                                mov        A, ADC2                ; Write the converted value in the accumulator
                                mov        B, #0x10                ; Divides the value by 16.
                                div        AB                    ; [0x00, 0xFF] -> [0x00, 0x0F]
                                mov        R6, A                  ; R6 stores the resulted value
                                mov        A, #0x0F
                                clr        C
                                subb      A, R6
                                mov        R7, A                  ; R7 = 0x0F - R6
                                ret

;-----
; End of file.

END

```